

Intelligenza Artificiale Distribuita

Svolgimento dell'elaborato.

Progetto scelto:

**Progetto 1: SISTEMA DI TRANSAZIONI BANCARIE DISTRIBUITE CON MUTUA
ESCLUSIONE A TOKEN RING**

Studente:

Michele De Rose - matr. 0322500130.

miderose@gmail.com

Link al repository di GitHub:

<https://github.com/miderose/MicheleDeRose-IntelligenzaArtificialeDistribuita>

- Repository pubblico.
- Contiene anche il video dei 4 terminali che eseguono il programma contemporaneamente.

Dettagli implementativi:

Linguaggio di programmazione utilizzato

Python. (v3.11)

Librerie o dipendenze richieste

Nessuna, usati solo moduli built-in di Python 3:
sys, socket, json, time, threading.

Modalità di compilazione (se applicabile)

Non applicabile: Python è interpretato.

Comandi esatti per avviare ciascuno dei 4 nodi

Eseguire `python atm_node.py <id_nodo>` per ogni nodo, ognuno, per apprezzare al meglio i messaggi di log che illustrano cosa sta succedendo, su un terminale diverso.

Esempio:

```
python atm_node.py 4
python atm_node.py 3
python atm_node.py 2
python atm_node.py 1
```

Il nodo 1 inietterà il token iniziale con saldo 1000, quindi è consigliabile farlo partire per ultimo, quando gli altri sono pronti.

Nei terminali (in tempo reale) e nel file `atm.log` saranno visibili i log di esecuzione.

Spiegazione del codice

Ho cercato di commentare il codice in modo che il file .py di implementazione della soluzione fosse auto-esplicativo, tuttavia riporto il principio di funzionamento di base:

- nella variabile globale **NODES_CONFIG** ci sono le configurazioni di ogni token:
 - la porta su cui sta in ascolto per la connessione del nodo precedente (definita come $5000 + \text{node_id}$, quindi 5001, 5002, ecc.);
 - la porta del nodo successivo a cui il nodo si collegherà;
 - le transazioni “pending”: le transazioni da eseguire quando si avrà il possesso del token.
- Il **main** del programma verifica gli argomenti (il *node_id*) con cui è stato avviato e la presenza dei dati del nodo in **NODES_CONFIG**. Superati questi controlli registra node come istanza della classe **ATMNode** a cui passa come parametro il *node_id*, dopodichè invoca il metodo *start* dell'istanza appena creata.
- **start()** è di fatto l'orchestratore delle connessioni:
 - avvia un *thread* che sta in ascolto per accettare la connessione del nodo precedente, poi delega a **accept_connection** la gestione dei messaggi in arrivo;
 - avvia la procedura per la connessione al nodo successivo (**connect_to_next**).
 - Poi sta in *loop infinito* (*while True*) con *sleep* di un secondo per tenere attivo il *thread* principale, fino alla pressione di *Ctrl-C* sulla tastiera, che termina il programma (ovviamente del solo nodo attuale).
 - Se il programma viene avviato come “nodo 1” allora dopo 5 secondi (per dare il tempo agli altri nodi di partire) viene iniettato nel “circuito” il **token**, che è un *json* con il “contatore” del saldo attuale.
- **accept_connection** accetta la connessione in ingresso sul *socket* e, quando arriva un messaggio, lo dirotta verso la funzione *handle_message* per farlo gestire.
- **connect_to_next** si collega con un *socket* al nodo successivo (protocollo: *IP (INET)*, tipo: *Stream*), che deve essere in ascolto; ha comunque un meccanismo di *retry infinito* fino a che non viene stabilita la connessione.
- **handle_message** gestisce i messaggi in arrivo sul *socket*:
 - si aspetta che un messaggio sia una stringa, terminata dal carattere di *new_line* (*\n*), che contenga un *Json*;
 - ottenuta la stringa del messaggio la converte quindi in un oggetto usando *json.loads*.
 - se il messaggio è di tipo *TOKEN* (nel nostro caso di esempio è l'unica tipologia di messaggio che circola...) allora delega alla funzione **process_token** le azioni da intraprendere sulla base di quanto richiesto nel messaggio.

- ***process_token*** è finalmente la nostra **sezione critica**: si occupa di eseguire la transazione sui dati “condivisi” (nel *token*):
 - si parte con uno *sleep*, che serve a far comparire il successivo messaggio di *log* dopo un secondo (utile per chi guarda i 4 terminali contemporaneamente...), si stampa il log informativo e poi si entra nel vivo con la transazione;
 - Se ci sono transazioni pending (quelle che abbiamo conservato in *NODES_CONFIG* per ogni nodo) allora, dopo i semplici controlli basilari del caso, si entra nel cuore:
 - se la transazione è un **prelievo** si controlla che il saldo sia sufficiente e poi da questo si decurta l'importo prelevato
 - se invece la transazione è di tipo **deposito** allora il saldo si aumenta dell'importo richiesto nella transazione.
 - si aggiorna quindi il saldo nel relativo campo del *token*, si stampano i *log* necessari e poi, dopo un ulteriore *sleep* di comodo per migliorare la comprensione di chi guarda i terminali, si inoltra finalmente il *token* con la funzione *forward_token*.
 - ***forward_token*** converte il *json* del *token* in stringa e poi lo invia nel *socket* collegato al nodo successivo.

A questo punto il lavoro del nostro ATM è, per questo turno, finito: quando il token compirà un giro completo “attraversando” tutti gli altri ATM e tornerà a noi allora l'elaborazione ricomincerà da capo e se ci saranno transazioni pending allora si tornerà ad eseguire la sezione critica, altrimenti si gira semplicemente il token al nodo successivo.

Michele De Rose.